Number: P3908R0
Date: 2025-12-14
Audience: Library Evolution
Target: C++29
Author: Hana Dusíková

- Motivation
- Implementation
  - Modern and fast algorithms
    - fast_float
    - Zmij
  - Compilation throughtput concerns
  - Alternative approach
- Wording
  - Feature test macro

Thanks to Daniel Lemire and Herb Sutter for nerd sniping me.

P3908R0: constexpr from_chars<float> / to_chars<

This paper makes float-to-chars and chars-to-float conversion functionality in <charconv> constexpr compatible.

Motivation

This is the missing piece of <charconv>. Its absence forces users to write their own conversions and that's against the purpose of standardization. By allowing it, we will be able to reuse this code in `constexpr` formatting using `std::format` too.

Implementation

The implementation of the conversion algorithms can be done `constexpr` compatible with current language without resorting to any magic. Implementation in libc++ needs to just move the code from .cpp files to headers. Same thing applies for libstdc++.

Modern and fast algorithms

fast_float

It's a library done by Daniel Lemire which implements proposed functionality fully in `constexpr`, and it has over 1.9 stars on Github, it's used in production code.

Zmij

Recently a modern approach named Zmij was discovered by Victor Zverovich, it's only ~1kLoC. Modifying it to be `constexpr` showed there is no performance difference. It uses only integer operations and `std::bit_cast`.

In terms of compile time performance of just including this code, the additional time to including this header is negligable.

Compilation throughtput concerns

If the complation performance is still a concern, especially for a big tables, this can be improved to delay the instantiation by making it a template which is instantiated when first used (including the constant evaluation of its values). This can be done by `template <typename = void>` trick, which lazily instantiate the table only if it's needed. This is used in fast_float library already.

Alternative approach

Alternative approach is to provide builtins doing the same operations. They can be implemented with `charconv` code itself. Or compiler can detect and bypass `from_chars` / `to_chars` instead of using explicit builtins.

Wording

The change is simple adding (as usual) a few of `constexpr` here and there.

28.2.1 Header <charconv> synopsis [charconv.syn]

1 When a function is specified with a type placeholder of *integer-type*, the implementation provides overloads for `char` and all cv-unqualified signed and unsigned integer types in lieu of *integer-type*. When a function is specified with a type placeholder of *floating-point-type*, the implementation provides overloads for all cv-unqualified floating-point types ([basic.fundamental]) in lieu of *floating-point-type*.

```
namespace std {
  // floating-point format for primitive numerical conversion
  enum class chars_format {
    scientific = unspecified,
    fixed = unspecified,
    hex = unspecified,
    general = fixed | scientific
  };

  // [charconv.to.chars], primitive numerical output conversion
```

```
  struct to_chars_result {
// freestanding
    char* ptr;
    errc ec;
    friend bool operator==(const to_chars_result&, const to_chars_
result&) = default;
    constexpr explicit operator bool() const noexcept { return ec =
= errc{}; }
  };

  constexpr to_chars_result to_chars(char* first, char* last,
// freestanding
                                     integer-type value, int base
= 10);
  to_chars_result to_chars(char* first, char* last,
// freestanding
                           bool value, int base = 10) = delete;

  constexpr to_chars_result to_chars(char* first, char* last,
// freestanding-deleted
                           floating-point-type value);
  constexpr to_chars_result to_chars(char* first, char* last,
// freestanding-deleted
                           floating-point-type value, chars_format
fmt);
  constexpr to_chars_result to_chars(char* first, char* last,
// freestanding-deleted
                           floating-point-type value, chars_format
fmt, int precision);

  // [charconv.from.chars], primitive numerical input conversion
  struct from_chars_result {
// freestanding
    const char* ptr;
    errc ec;
    friend bool operator==(const from_chars_result&, const from_ch
ars_result&) = default;
    constexpr explicit operator bool() const noexcept { return ec =
= errc{}; }
  };

  constexpr from_chars_result from_chars(const char* first, const
char* last,    // freestanding
                                         integer-type& value, in
t base = 10);
```

```
  constexpr from_chars_result from_chars(const char* first, const
char* last,      // freestanding-deleted
                                         floating-point-type& value,
                                         chars_format fmt = chars_format::g
eneral);
}
```

2 The type `chars_format` is a bitmask type ([bitmask.types]) with elements `scientific`, `fixed`, and `hex`.

3 The types `to_chars_result` and `from_chars_result` have the data members and special members specified above. They have no base classes or members other than those specified.

28.2.2 Primitive numeric output conversion [charconv.to.chars]

1 All functions named `to_chars` convert `value` into a character string by successively filling the range `[first, last)`, where `[first, last)` is required to be a valid range. If the member `ec` of the return value is such that the value is equal to the value of a value-initialized `errc`, the conversion was successful and the member `ptr` is the one-past-the-end pointer of the characters written. Otherwise, the member `ec` has the value `errc::value_too_large`, the member `ptr` has the value `last`, and the contents of the range `[first, last)` are unspecified.

2 The functions that take a floating-point `value` but not a `precision` parameter ensure that the string representation consists of the smallest number of characters such that there is at least one digit before the radix point (if present) and parsing the representation using the corresponding `from_chars` function recovers `value` exactly.

[*Note 1*: This guarantee applies only if `to_chars` and `from_chars` are executed on the same implementation. — *end note*]

If there are several such representations, the representation with the smallest difference from the floating-point argument value is chosen, resolving any remaining ties using rounding according to `round_to_nearest` ([round.style]).

3 The functions taking a `chars_format` parameter determine the conversion specifier for `printf` as follows: The conversion specifier is `f` if `fmt` is `chars_format::fixed`, `e` if `fmt` is `chars_format::scientific`, `a` (without leading `"0x"` in the result) if `fmt` is `chars_format::hex`, and `g` if `fmt` is `chars_format::general`.

```
constexpr to_chars_result to_chars(char* first, char* last, integer-
type value, int base = 10);
```

4 *Preconditions*: `base` has a value between 2 and 36 (inclusive).

5

Effects: The value of `value` is converted to a string of digits in the given base (with no redundant leading zeroes). Digits in the range 10..35 (inclusive) are represented as lowercase characters a..z. If `value` is less than zero, the representation starts with `'-'`.

6 *Throws*: Nothing.

```
constexpr to_chars_result to_chars(char* first, char* last, floating-point-type value);
```

7 *Effects*: `value` is converted to a string in the style of `printf` in the `"C"` locale. The conversion specifier is `f` or `e`, chosen according to the requirement for a shortest representation (see above); a tie is resolved in favor of `f`.

8 *Throws*: Nothing.

```
constexpr to_chars_result to_chars(char* first, char* last, floating-point-type value, chars_format fmt);
```

9 *Preconditions*: `fmt` has the value of one of the enumerators of `chars_format`.

10 *Effects*: `value` is converted to a string in the style of `printf` in the `"C"` locale.

11 *Throws*: Nothing.

```
constexpr to_chars_result to_chars(char* first, char* last, floating-point-type value,
                                   chars_format fmt, int precision);
```

12 *Preconditions*: `fmt` has the value of one of the enumerators of `chars_format`.

13 *Effects*: `value` is converted to a string in the style of `printf` in the `"C"` locale with the given precision.

14 *Throws*: Nothing.

See also: ISO/IEC 9899:2024, 7.23.6.2

28.2.3 Primitive numeric input conversion [charconv.from.chars]

1 All functions named `from_chars` analyze the string `[first, last)` for a pattern, where `[first, last)` is required to be a valid range. If no characters match the pattern, `value` is unmodified, the member `ptr` of the return value is `first` and the member `ec` is equal to `errc::invalid_argument`.

[*Note 1*: If the pattern allows for an optional sign, but the string has no digit characters following the sign, no characters match the pattern. — *end note*]

Otherwise, the characters matching the pattern are interpreted as a representation of a value of the type of `value`. The member `ptr` of the return value points to the first character not matching the pattern, or has the value `last` if all characters

match. If the parsed value is not in the range representable by the type of `value`, `value` is unmodified and the member `ec` of the return value is equal to `errc::result_out_of_range`. Otherwise, `value` is set to the parsed value, after rounding according to `round_to_nearest` ([round.style]), and the member `ec` is value-initialized.

```
constexpr from_chars_result from_chars(const char* first, const char*
last,
                                       integer-type& value, int base =
10);
```

2 *Preconditions*: `base` has a value between 2 and 36 (inclusive).

3 *Effects*: The pattern is the expected form of the subject sequence in the `"C"` locale for the given nonzero base, as described for `strtol`, except that no `"0x"` or `"0X"` prefix shall appear if the value of `base` is 16, and except that `'-'` is the only sign that may appear, and only if `value` has a signed type.

4 *Throws*: Nothing.

```
constexpr from_chars_result from_chars(const char* first, const char*
last, floating-point-type& value,
                               chars_format fmt = chars_format::gener-
al);
```

5 *Preconditions*: `fmt` has the value of one of the enumerators of `chars_format`.

6 *Effects*: The pattern is the expected form of the subject sequence in the `"C"` locale, as described for `strtod`, except that

(6.1) — the sign `'+'` may only appear in the exponent part;

(6.2) — if `fmt` has `chars_format::scientific` set but not `chars_format::fixed`, the otherwise optional exponent part shall appear;

(6.3) — if `fmt` has `chars_format::fixed` set but not `chars_format::scientific`, the optional exponent part shall not appear; and

(6.4) — if `fmt` is `chars_format::hex`, the prefix `"0x"` or `"0X"` is assumed.

[*Example 1*: The string `0x123` is parsed to have the value `0` with remaining characters `x123`. — *end example*]

In any case, the resulting `value` is one of at most two floating-point values closest to the value of the string matching the pattern.

7 *Throws*: Nothing.

See also: ISO/IEC 9899:2024, 7.24.2.6, 7.24.2.8

Feature test macro

17.3.2 Header <version> synopsis [version.syn]

```
#define __cpp_lib_constexpr_charconv 202207L20????L          // // freestanding, al
so in <charconv>
```